

Grafy, algorytmy przeglądu

Definicje, oznaczenia

// " := " oznacza "z definicji"

Graf skierowany (zorientowany) $G=(V, E) :=$

- V - zbiór wierzchołków (węzłów), $0 < |V| < \infty$;
- $E \subseteq V \times V$ - zbiór krawędzi.
(krawędzie to uporządkowane pary wierzchołków)

Jeśli $(u, v) \in E$, to

- wierzchołek v jest następnikiem wierzchołka u ,
- wierzchołek u jest poprzednikiem wierzchołka v ,
- krawędź (u,v) jest incydentna z wierzchołkami u, v ,
 u, v są incydentne z krawędzią (u,v) .

Graf nieskierowany (niezorientowany) $G=(V, E) :=$

- V - zbiór wierzchołków (węzłów), $0 < |V| < \infty$;
- $E \subseteq 2^V$ - zbiór krawędzi, $\forall e \in E : |e| = 2$,
(krawędzie to dwuelementowe podzbiory)

Jeśli $\{u, v\} \in E$, to

- wierzchołek u jest sąsiadem wierzchołka v ,
- krawędź $\{u,v\}$ jest incydentna z wierzchołkami u, v
i na odwrót;

krawędź $\{u, v\}$ często zapisujemy jako parę (u, v) .

Graf ważony (skierowany, nieskierowany)

$G=(V, E, w) :=$

- $w : V \rightarrow \mathbb{R}$ - funkcja wagowa krawędzi,
- $w(u,v)$ - waga (koszt) krawędzi (u,v) .

Typowe oznaczenia rozmiarów grafu:

$$|V| = n = V, \quad |E| = m = E.$$

Fakt.

Liczba krawędzi m w grafie o n wierzchołkach wynosi:

- w grafie skierowanym: $0 \leq m \leq n \cdot n$
- w grafie nieskierowanym: $0 \leq m \leq n \cdot (n-1)/2$

Terminologia

- stopień wierzchołka (graf nieskierowany) :=
liczba jego sąsiadów;
- stopień wyjściowy wierzchołka (graf skierowany) :=
liczba jego następników;
- stopień wejściowy wierzchołka (graf skierowany) :=
liczba jego poprzedników;
- ścieżka z wierzchołka u do wierzchołka w :=
ciąg wierzchołków $\langle v_0, v_1, \dots, v_k \rangle$ taki, że
 $(v_i, v_{i+1}) \in E, i=0, \dots, k-1, v_0 = u, v_k = w$;
- w jest osiągalny z wierzchołka u :=
istnieje ścieżka z u do w ;
- długość ścieżki :=
liczba krawędzi na ścieżce (tutaj: k);
- ścieżka prosta (elementarna) :=
wierzchołki na ścieżce nie powtarzają się;
- cykl := ścieżka postaci $\langle v_0, v_1, \dots, v_k=v_0 \rangle$,
dodatkowo, dla grafów nieskierowanych na
cyklu muszą być co najmniej 3 krawędzie;
- cykl prosty (elementarny) :=
cykl w którym wierzchołki nie powtarzają się,
za wyjątkiem $v_0=v_k$;
- odległość wierzchołka v od wierzchołka u :=
długość najkrótszej ścieżki z u do v
(∞ jeśli v nieosiągalny z u);

- podgraf grafu $G=(V, E) :=$
graf $H=(V', E')$ taki, że $V' \subseteq V, E' \subseteq E$;
- podgraf indukowany grafu $G=(V, E) :=$
graf $H=(V', E')$ taki, że $V' \subseteq V$,
 $E' = \{(u,v) \in E \mid u,v \in V'\}$.

Dla grafu nieskierowanego $G=(V, E)$:

- G jest spójny $:=$
każdą parę wierzchołków łączy jakaś ścieżka;
- spójna składowa grafu $G :=$
maksymalny (w sensie inkluzji) podgraf
indukowany H taki, że H jest spójny;
- G jest drzewem $:=$
 G jest spójny i nie zawiera cykli (często z
wyróżnionym wierzchołkiem zwanym
korzeniem - drzewo ukorzenione);
- drzewo rozpinające dla spójnego nieskierowanego
grafu $G=(V,E) :=$
drzewo $T=(V', E')$ takie, że $V'=V, E' \subseteq E$
(czyli podgraf spójny bez cykli zawierający
wszystkie wierzchołki).

Dla grafu skierowanego:

- G jest silnie spójny $:=$
gdy każdy wierzchołek jest osiągalny z
każdego wierzchołka;
- silnie spójna składowa grafu $G :=$
maksymalny (w sensie inkluzji) podgraf
indukowany H taki, że H jest silnie spójny.

Reprezentacja – struktura danych

Utożsamiamy: $V = \{1, \dots, n\}$

1. Tablica sąsiedztwa:

- tablica $A[1..n, 1..n]$
$$A[i, j] = \begin{cases} 1, & \text{jeśli } (i, j) \in E \\ 0, & \text{jeśli } (i, j) \notin E \end{cases}$$
- jej wielkość nie zależy od liczby krawędzi w grafie;
- dla grafu nieskierowanego jest symetryczna, z zerami na przekątnej;
- wielkość zajmowanej pamięci: $\Theta(n^2)$.

2. Listy sąsiedztwa (listy następników):

- tablica $Adj[1..n]$ (nagłówków) list;
- $Adj[u]$ jest nagłówkiem listy zawierającej wszystkie wierzchołki v takie że $(u, v) \in E$;
- kolejność na listach dowolna;
- dla grafu nieskierowanego każda krawędź (u, v) reprezentowana jest w liście $Adj[u]$ oraz w liście $Adj[v]$;
- wielkość zajmowanej pamięci: $\Theta(n+m)$;
- praktycznie: listy sąsiedztwa często realizuje się jako tablice dynamiczne (np. *vector* w STL).

Tablica sąsiedztwa:

- zaleta: dla zadanych u, v , test $(u, v) \in E$ w czasie $O(1)$;
- wada: pamięć $\Theta(n^2)$

Listy sąsiedztwa:

- zaleta: pamięć $\Theta(n+m)$, istotna dla dużych rzadkich grafów.

Przegląd grafu

możemy zdefiniować jako zadanie następujące:

dla ustalonego wierzchołka s , oblicz

$$R(s) = \{v: \text{istnieje ścieżka z } s \text{ do } v\}$$

("v reachable from s")

Zauważmy:

G – nieskierowany $\Rightarrow$

$R(s)$ jest spójną składową zawierającą s

Algorytm "generyczny" przeglądu grafu:

```
Search(G,s)
// P - tablica indeksowana wierzchołkami,
// początkowo P[v]=NULL, dla każdego v
  R := {s};
  while istnieje (u,v) ∈ E(G) : u ∈ R, v ∉ R do
    R := R ∪ {v};
    P[v] := u;
  return R,P;
end.
```

Fakt:

Założmy, że

- G - graf nieskierowany, $v \in V(G)$,
- wykonano $\text{Search}(G,v)$,
- $T = (V(G), E')$, $E' = \{\{u,v\} : u,v \in V(G), u = P[v]\}$.

Wówczas T jest drzewem rozpinającym dla $R(v)$.

Dowód: łatwa indukcja. OK.

Efektywna implementacja: możliwe różne rozwiązania,
w szczególności BFS i DFS.

BFS – przeglądanie grafu wszerz (breadth-first search)

Startujemy z zadanego wierzchołka startowego s , odwiedzamy:

- najpierw wierzchołki odległe od wierzchołka startowego o 1,
- następnie wierzchołki odległe o 2,
- następnie wierzchołki odległe o 3, itd.

Podstawa realizacji: kolejka.

Zawiera:

- węzły do których już dotarliśmy, pokolorowane na kolor szary,
- ale dla których jeszcze nie badaliśmy możliwych wyjść (krawędzi) prowadzących dalej.

Tablica $col[1..n]$, $col[v]$ - kolor wierzchołka:

- początkowo wszystkie wierzchołki białe;
- wierzchołek wstawiony do kolejki: szary;
- usunięty z kolejki: czarny.

Tablica $par[1..n]$, $par[v]$ - poprzednik węzła v w konstruowanym drzewie rozpinającym;

Tablica $dist[1..n]$, $dist[v]$ - odległość v od źródła s .

```

Algorytm BFS(G, s)
// przegląd wszerz grafu G, wierzchołek startowy s
// col[], dist[], par[] (color, distance, parent)
//   - tablice indeksowane wierzchołkami
// Q - kolejka wierzchołków
// założenie: wszystkie wierzchołki osiągalne z s
// zapis algorytmu uniwersalny:
//   - dla grafów skierowanych i nieskierowanych

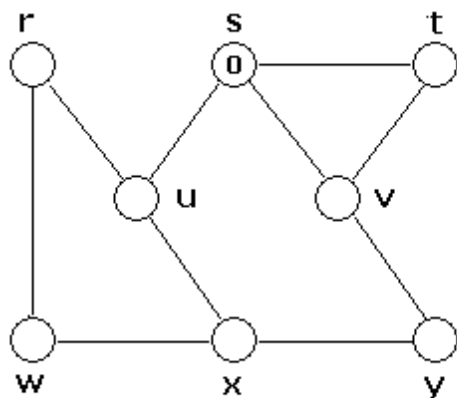
  for each  $u \in V(G) - \{s\}$  do
    col[u] := white;
  col[s] := grey;
  dist[s] := 0;
  par[s] := NULL;

  Q.Create;
  Q.Insert(s);

  while Q.nonempty do
    u := Q.First;
    for each  $v \in \text{Adj}[u]$  do
      if col[v] = white then
        col[v] := grey;
        dist[v] := dist[u] + 1;
        par[v] := u;
        Q.Insert(v);
    col[u] := black;
    Q.DeleteFirst;

end.

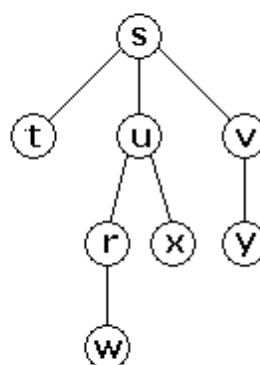
```



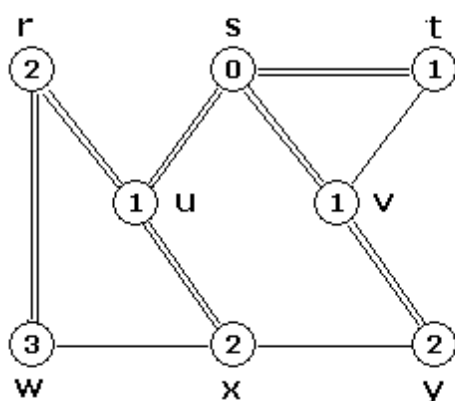
Kolejność wstawiania do kolejki:

s t u v r x y w

Drzewo rozpinające:



Obliczone odległości:



Własności BFS-u

Dowody poniższych faktów są dość łatwe, na ogół przez indukcję.

Fakt 1.

BFS jest poprawną implementacją algorytmu generycznego, czyli:

- odwiedza każdy wierzchołek osiągalny z wierzchołka s,
- generuje drzewo rozpinające spójnej składowej zawierającej s (w przypadku drzew nieskierowanych).

Fakt 2.

Dla każdego wierzchołka u w grafie, obliczona przez algorytm wielkość $\text{dist}[u]$ jest odległością u od s.

Definicja.

Drzewo rozpinające grafu G o korzeniu s , w którym dla każdego wierzchołka v ścieżka od s do v jest najkrótszą ścieżką nazywamy drzewem przeszukiwania wszerz (drzewem BFS) grafu G .

Fakt 3.

Drzewo generowane przez algorytm BFS jest drzewem przeszukiwania wszerz o korzeniu s .

Fakt 4.

W dowolnej chwili, jeśli wierzchołki v i w są w kolejce Q , oraz w został wstawiony do Q później niż v , to
$$d[v] \leq d[w] \leq d[v] + 1.$$

Fakt 4'.

Dokładniej, w dowolnym momencie, na początku kolejki są wierzchołki odległe od źródła o k , dla pewnego $k \geq 0$, a za nimi w kolejce są wierzchołki odległe o $k+1$, i nie ma innych wierzchołków.

Fakt 5. (nierówność trójkąta, niezależna od BFS)

Dla każdej krawędzi $(u,v) \in E(G)$, dla dowolnie ustalonego $s \in V(G)$, $d(s,v) \leq d(s,u) + 1$.

Ćwiczenie:

jaki będzie wynik BFS jeśli na początku do kolejki wstawimy i pokolorujemy na szaro nie jeden, lecz więcej wierzchołków, np. 2?

Złożoność algorytmu BFS

- każdy wierzchołek v wstawiany i usuwany z kolejki dokładnie raz,
- $\Rightarrow$ liczba iteracji pętli *while* wynosi n ;
- każda krawędź wychodząca z wierzchołka badana dokładnie raz,
- $\Rightarrow$ liczba iteracji pętli *while* wynosi n ;
- $\Rightarrow$ liczba iteracji pętli *for each* $v \in Adj[u]$ w sumie we wszystkich iteracjach pętli *while* jest równa sumie długości list sąsiedztwa:
 - $2m$ jeśli graf nieskierowany,
 - m jeśli graf skierowany.

Zatem całość: $\Theta(n+m)$.

Uwaga1

- w praktyce najczęstszym zastosowaniem BFS-u jest obliczanie odległości od zadanego źródła;
- do tego celu wystarcza kolorować drugim kolorem wierzchołek wstawiany do kolejki;
- trzeci kolor oraz tablica poprzedników są wtedy niepotrzebne.

Uwaga 2

- gdy graf nieskierowany jest niespójny,
albo
- w grafie skierowanym nie istnieje wierzchołek s z którego wszystkie inne wierzchołki są osiągalne,
a potrzebujemy odwiedzić każdy wierzchołek, to:
- należy wywoływać procedurę BFS "do skutku",
- tzn. w pętli zewnętrznej wyszukiwać dowolny dotychczas nieodwiedzony wierzchołek i rozpoczynać od niego przegląd.

```

Algorytm BFS(G)
// przegląd całego grafu, bez założenia o spójności
// wersja bazowej BFS:
//     - tylko znakujemy wierzchołki odwiedzone,
// nie obliczamy odległości ani drzewa rozpinającego

    for each  $x \in V(G)$  do
        visited[x] := false;
    Q.Create;
    for each  $x \in V(G)$  do
        if not visited[x] then
            visited[x] := true;
            Q.Insert(x);
            while Q.nonempty do
                 $u := Q.DeleteFirst$ ;
                for each  $v \in Adj[u]$  do
                    if not visited[v] then
                        visited[v] := true;
                        Q.Insert(v);
            end.
end.

```